

Docker

Introduction to Docker

Plan

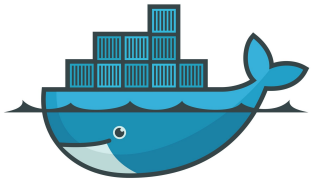
1. **Docker**
2. **Key concepts**
   a. Image
   b. Dockerfile
   c. Layer
   d. Container
   e. Register
   f. Architecture
3. **Useful commands**
4. **More advanced concepts**
5. **In practice**
   a. Docker Desktop
   b. Dockercoins

https://ai-training.fr/



I. Introduction

Introduction
Docker



Docker is a platform for containerization which enables applications to be developed, tested and deployed in containers.

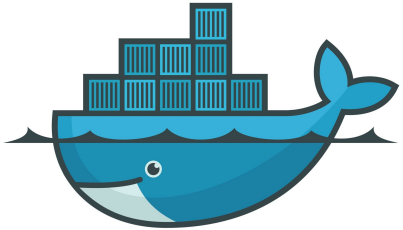
The advantages of Docker :

- **Insulation each application is ran in an isolated container, avoiding dependency conflicts and compatibility problems.**
- **Portability docker containers are portable and can be run on any machine that supports Docker.**
- **Ease of deployment docker: simplifies application deployment, as all you have to do is deploy the Docker container on the target server.**
- **Speed the new "Runtime:" feature lets you quickly launch runtime environments for applications, accelerating development and deployment cycles.**

https://ai-training.fr/

Introduction

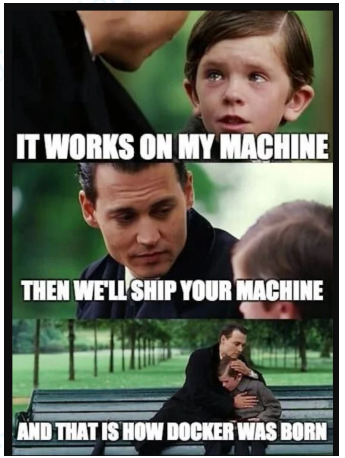
Who uses Docker



Docker has become an indispensable tool in modern software development and IT infrastructure.

It is used by developers to create runtime environments for applications by system administrators to manage server infrastructures, by the teams DevOps to automate application deployment and, of course, MLOps!

->Docker is the answer to the recurring problem of "the code works fine on my machine"

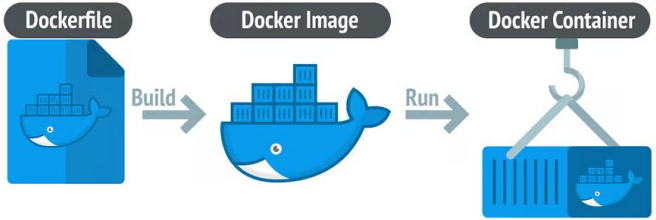




II. Key concepts

Key concepts

The image

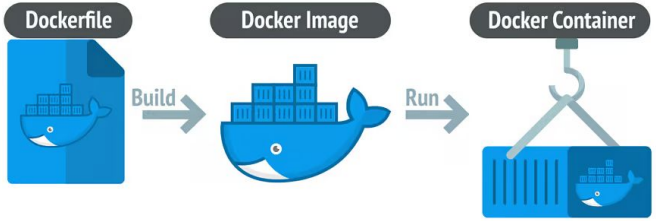


- **An image is a read-only "template" containing instructions for creating a Docker container.**
- **Often, an image is based on another image (but it may not be), with a few additional customizations.**
- **For example, you could create an image based on the ubuntu image, but which installs the Apache web server and your application, as well as the configuration details needed to run your application.**
- **You can create your own images with a Dockerfile or use only those created by others and published in a register.**

https://ai-training.fr/

Key concepts

The Dockerfile



- **To create your own image, you create a Dockerfile with a simple syntax for defining the steps required to create and execute the image.**
- **Each instruction in a Dockerfile creates a layer in the image.**
- **When you modify the Dockerfile and rebuild the image, only layers that have been modified are rebuilt (cf: cache). This is part of what makes images so light, small and fast compared to other virtualization technologies.**

```
# It instructs Docker Engine to use official
# python:3.10 as the base image
FROM python:3.10

# It creates a working directory(app) for the
# Docker image and container
WORKDIR /app

# It copies the framework and the dependencies for
# the FastAPI application into the working directory
COPY requirements.txt .

# It will install the framework and the dependencies
# in the `requirements.txt` file.
RUN pip install -r requirements.txt

# It will copy the remaining files and the source code
# from the host `fast-api` folder to the `app` container
# working directory
COPY . .

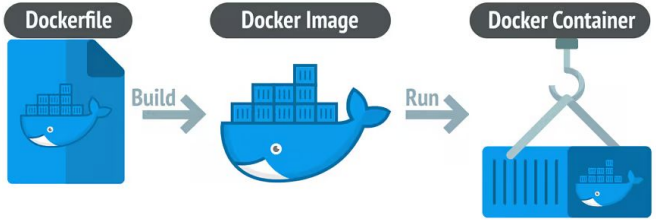
# It will expose the FastAPI application on port `8000`
# inside the container
EXPOSE 8000

# It is the command that will start and run the FastAPI
# application container
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

https://ai-training.fr/

Key concepts

The Layer



Each layer will add or modify files

Interests :

- **reuse / share layers**
- **save money :**
  - disk space
  - network transfer
  - memory
- **they are read-only**

Layers are organized via the Dockerfile

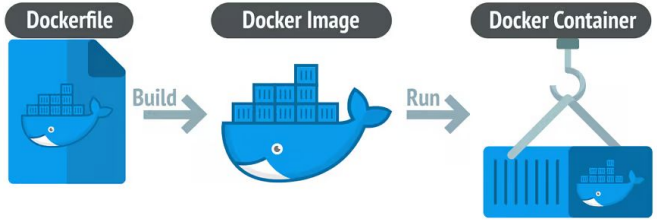
"Layers are like glass plates with files engraved on them:
seen from above, the plates are stacked:
all the engravings are transparent.

The layers are read-only, I can't modify them, except for the last plate which is accessible"

https://ai-training.fr/

Key concepts

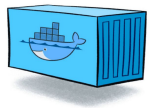
The container



Docker containers are lightweight, portable runtime environments that contain all software dependencies required to run an application.

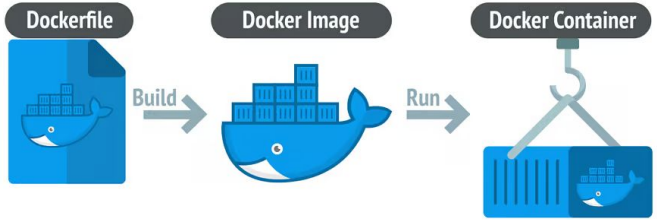
A container is a instance image executable. You can create, start, stop, move or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage or even create a new image based on its current state.

Almost nothing runs on the container (ps command)
Every time we make a docker run we'll create a new container
- Advantages: the container can be destroyed without risk
- Disadvantages: what you do in a container will only be done in that container
- Why a new container? To encourage repeatability/automation of deployments



Key concepts

The container

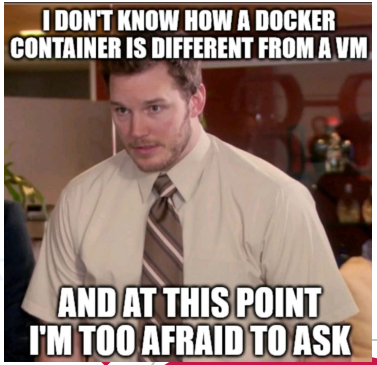


Why use containers?

- Containers have no operating system and use the host operating system. They share libraries and resources as required.
- Insulation applications: If you want to run several applications on the same server, keeping the components of each application in separate containers avoids problems related to dependency management.
- Application processing and execution very fast since binaries and libraries specific to container applications run on the host kernel.
- Starting up a container takes just a fraction of a second.
- Containers are lighter and faster than virtual machines.

You can think of it as OOP:

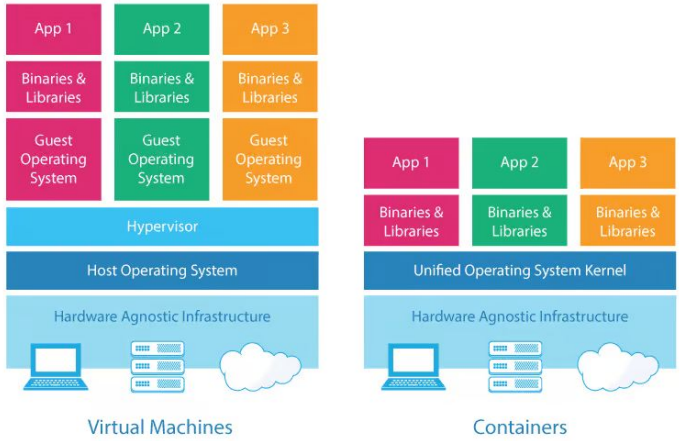
Class -> image
Instance -> container
Inheritance -> add layers



https://ai-training.fr/

The container VS virtual machine

- **Virtual machines generally include a complete operating system packages and, where applicable, a few applications.**
- **Docker is a platform that groups together an application and all its dependencies in the form of containers.**
  Instead of hosting a operating system by application, certain common resources can be shared docker Engine, which sits on top of an operating system.
- **Containers are similar to virtual machines, except that they are not complete operating systems. Containers generally only include the packages and applications required by the operating system.**
  They generally do not contain a complete operating system or hardware virtualization: they are "light".



To sum up:
VM: work with a disk image (takes up space)
Container: we have an archive with files

Key concepts

The register



A Docker registry stores the Docker images.

Docker Hub is a public register that anyone can use, and Docker is configured to search for images on Docker Hub by default. You can even manage your own private registry.

We have :

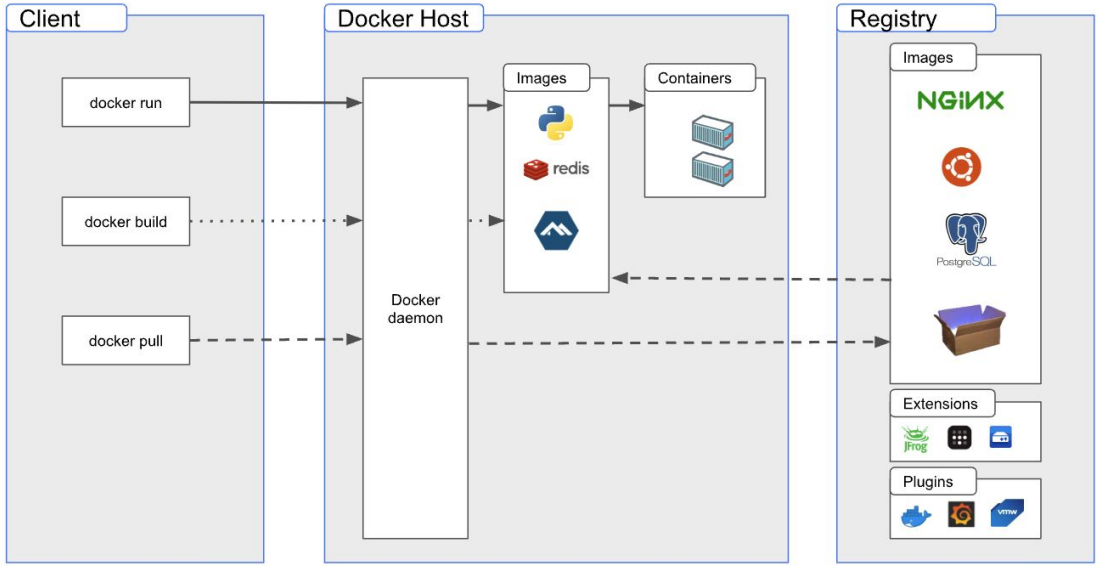
- **official images: around a hundred (e.g. redis). It's best to use them whenever possible!**
- **Personal images :**
  - Create a Docker Hub account (e.g. jpetazzo/clock)
  - Or host our own registry: we'll have the registry name in the image (e.g. quay.io/correos/etcd)

https://ai-training.fr/

Key concepts

Architecture

- **Visit docker daemon (dockerd) listens to requests from the Docker API and manages Docker objects such as images the containers the networks and volumes. A daemon can also communicate with other daemons to manage Docker services.**
- **Visit docker client is the main way in which many Docker users interact with Docker.**
- **When you use commands such as docker run the customer sends these orders to dockerd which executes them. The docker command uses the Docker API.**
  The Docker client can communicate with several daemons.





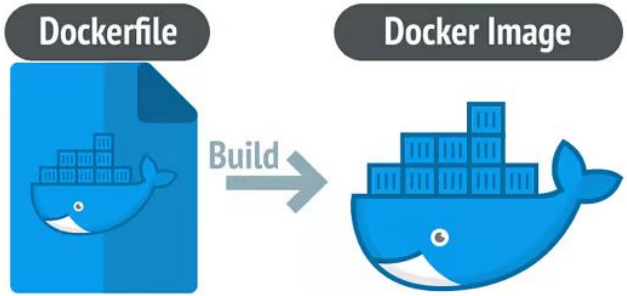
III. Useful commands



Useful commands

Docker build

When you build an image, Docker will rely on the Dockerfile and build the image.



Example: docker build . -t image_name:tag

It will use the cache this is why it's important to optimize your instructions in the Dockerfile to create a good level of abstraction for the layers and to order them correctly.

https://ai-training.fr/

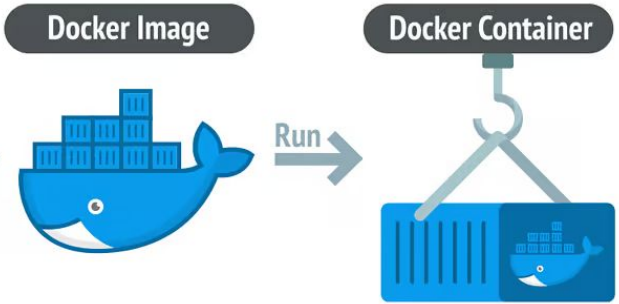
Useful commands

Docker run

The Docker run command allows you to launch a container (an instance) of an image.

Example: docker run busybox echo hello world

- **docker run: command to launch a container**
- **busybox: name of image to be launched**
- **echo hello world : command to execute**



It will fetch and download (*sweater*) the image if it doesn't exist locally.

If you re-run the command, the image will already be there, so execution will be much faster.

-it: interactive (-i: i/o to connect to the container, -t: to enter a pseudo-terminal)

-d : detach / demonize to turn it into a daemon -> returns the container identifier



Useful commands

How to create a new image

The classic: Dockerfile then docker build

But you can also do :

- **docker commit**
  - By default for each line of the **Dockerfile** it adds a **layer**. Whereas if you use docker commit, you'll have a single layer that encapsulates everything in that commit.
- **docker import:**
  - allows you to take a TAR archive and make an image of it, useful when you're starting from scratch
- **FROM scratch in the Dockerfile**

Example of DockerFile with figlet on ubuntu version 20.04 :

```
FROM ubuntu:20.04
RUN apt update
RUN apt install figlet
```

Then with docker build (I call the image tag '20.04') :

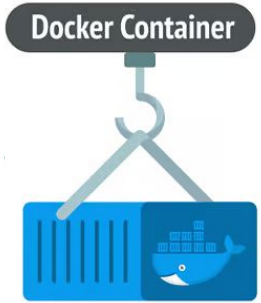
```
docker build . -t figlet:20.04
```

You can test it by launching an interactive container with docker run :

```
docker run -it figlet:20.04
```

Useful commands

Docker exec



- Docker exec lets you run any command in the container (a bit like ssh).

  Especially suitable for debug.

- Disadvantage: the program to be launched must be in the container!

  Example to view processes: docker exec <id_container> ps -false

- Most of the time, we'll be working with containers WITHOUT using docker exec :
  - view logs: docker logs
  - package : Dockerfile
  - metrics: Prometheus

https://ai-training.fr/

Useful commands

Other useful commands

There are many other commands:

- docker **stop** to stop a container by sending a signal to the first process in this container. After 10 seconds, the docker engine sends the KILL signal.
- docker **kill**: kills the container directly (KILL signal). A killed container consumes no more memory, but only a tiny amount of disk space. It can therefore be restarted if necessary.
- docker **diff**: Shows changes made with docker commit, like a commit on git
- docker **inspect**: returns a rather long JSON about the container
- docker **logs**: useful for viewing container logs
- docker **ps**: to see the containers
- etc.

https://ai-training.fr/



IV. Advanced concepts

Advanced concepts

The network

We have networks to connect containers on these different networks.

With docker network networks can be created and managed.

You can also specify the network with --net in docker run

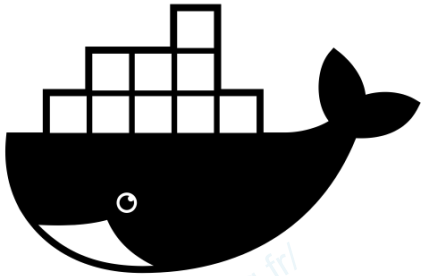
Example: docker run --net dev -ti alpine => provides isolation between networks



https://ai-training.fr/

Advanced concepts

The cache



- Restarts the builder from the first layer to be modified.
  Previous layers are cached.

- The cache makes it possible to build complex, repeatable builds that are also fast

How does it work?

Comparison of single strings on Dockerfile lines
Example: if I add a space to the command :

- Docker will not take the cache and will rebuild the Layer because it considers that there has been a change
- It will then rebuild the layers of all the lines below it

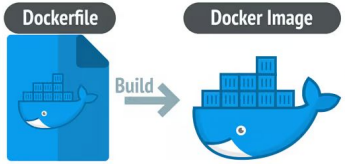
How to get rid of the cache?

In the docker build :

- use ``--no-cache``: to avoid using the cache
- use ``--pull": Docker goes to the base image and re-downloads it intelligently, thus rebuilding all the layers applied to it

Advanced concepts

Build optimization



Dependency management
In the case of dependencies (e.g. pip install -r requirements.txt) :

- **first install the dependencies**
- **And then just copy the code (COPY)**

Linking commands in a single layer, when?
You can also chain commands to put them in the same layer (because when you do a docker pull, it downloads all the layers, a bit like the git history).

- **If this docker is going to be pulled/pushed billions of times, we optimize.**
- **Otherwise leave it as it is, it's clearer with several lines per order**

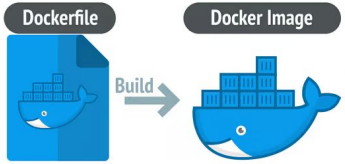
Factor information with environmental variables (ENV)

Reduce image size
If I have to pull/push a particular layer many times, I need to optimize it.
Otherwise, it's the max image size that will be interesting to optimize.

https://ai-training.fr/

Advanced concepts

Image size optimization



1. multi-stage build the first and most efficient technique. Consists in making several FROM entries in the Dockerfile.

Each FROM statement can use a different base, and each starts a new build step. You can copy **selectively** artifacts from one step to the next, leaving behind anything you don't want in the final image.

The second FROM instruction starts a new construction step with the alpine:latest image as a base. The line **COPY --from=builder** copies only the artifact built in the previous step into this new step.
The Go SDK and all intermediate artifacts are left behind, and are not saved in the final image.

```
# syntax=docker/dockerfile:1

FROM golang:1.16 AS builder
WORKDIR /go/src/github.com/alexellis/href-counter/
RUN go get -d -v golang.org/x/net/html
COPY app.go ./
RUN CGO_ENABLED=0 go build -a -installsuffix cgo -o app .

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /go/src/github.com/alexellis/href-counter/app ./
CMD ["./app"]
```

2. images Alpine light image

3. images slim light image

4. images FROM scratch building an image from scratch

Advanced concepts

Compose



Docker compose allows you to launch several containers that cooperate with each other (e.g. launch a Stack ELK).

After editing a docker-compose.yaml file, we can :

- **docker-compose build to reconstruct container images**
- **docker-compose up to restart the stack with the new images**

We can also combine the two with docker-compose up --build

```yaml
version:"3"
services:
 www:
   build www
   ports:
     - ${PORT-8000}:5000
   user: nobody
   environment:
     DEBUG:1
   command python counter.py
   volumes:
     - ./www:/src
 redis:
   image: redis
```

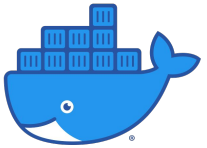
https://ai-training.fr/



V. In practice

In practice

Installing Docker Desktop



Docker Desktop is an easy-to-install application for your Mac, Windows or Linux environment, enabling you to create and share containerized applications and microservices.

Docker Desktop includes the Docker daemon (dockerd), the Docker client (docker), Docker Compose and Kubernetes.

Download link: https://www.docker.com/products/docker-desktop/

https://ai-training.fr/

In practice

The Dockercoins application

This is a fictitious application created by https://github.com/jpetazzo

The aim of the application is to simulate cryptocurrency mining using 5 services:

- **rng = web service generating random bytes**
- **hasher = web service that calculates data hashes**
- **worker = background process calling rng and hasher**
- **webui = web interface to monitor progress**
- **redis = data store (contains a counter updated by worker)**

In practice :
In an infinite loop, the worker calls the rng web service to generate random bytes and calls the `hasher` web service to hash these bytes.
Every second, the worker updates redis to indicate how many loops have been completed. The webui queries redis, calculates and displays the "hash speed" in your browser.

